

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
Факультет Программной инженерии и компьютерной техники

Системы искусственного интеллекта
Лабораторная работа №5

Выполнил студент
Егорова Варвара Александровна
Группа Р3323
Преподаватель: Королева Юлия Александровна

Содержание

Текст задания.....	3
Реализация.....	4
Эксперименты.....	9
Вывод.....	13
Приложение.....	14

Текст задания

Необходимо реализовать простую нейронную сеть (не обязательно свёрточную) и обучить её на датасете. Каждый слой прописать вручную. В классе не должно быть встроенных методов `torch`. Вывести график нескольких моделей на экспериментах, выбрать лучшую модель и протестировать её на своих данных.

Реализация

Далее на листингах 1-7 представлена реализация классов разных слоёв нейронной сети, “контейнера” модели, оптимизатора и цикла обучения.

Листинг 1 - Реализация класса слоя Linear.

```
import torch

class Linear:
    def __init__(self, in_features, out_features):
        self.W = torch.randn(in_features, out_features) * 0.01
        self.b = torch.zeros(out_features)

        self.dW = torch.zeros_like(self.W)
        self.db = torch.zeros_like(self.b)

    def forward(self, X):
        self.X = X
        return X @ self.W + self.b

    def backward(self, grad_out):
        self.dW = self.X.T @ grad_out
        self.db = grad_out.sum(dim=0)

        return grad_out @ self.W.T
```

Листинг 2 - Реализация класса слоя ReLU.

```
import torch

class ReLU:
    def __init__(self):
        pass

    def forward(self, x):
        self.mask = x > 0
        return x * self.mask

    def backward(self, grad_out):
        return grad_out * self.mask
```

Листинг 3 - Реализация класса слоя Flatten.

```
import torch
```

```

class Flatten:
    def __init__(self):
        pass

    def forward(self, x):
        self.input_shape = x.shape
        batch_size = x.shape[0]
        return x.view(batch_size, -1)

    def backward(self, grad_out):
        return grad_out.view(self.input_shape)

```

Листинг 4 - Реализация класса слоя SoftMax+CrossEntropy.

```

import torch

class SoftmaxCrossEntropy:
    def __init__(self):
        pass

    def forward(self, logits, y_true):
        shifted_logits = logits - logits.max(dim=1, keepdim=True).values
        exp_logits = torch.exp(shifted_logits)
        probs = exp_logits / exp_logits.sum(dim=1, keepdim=True)

        self.probs = probs
        self.y_true = y_true
        self.batch_size = logits.shape[0]

        log_probs = torch.log(probs + 1e-9)
        loss = -log_probs[torch.arange(self.batch_size), y_true].mean()

        return loss

    def backward(self):
        grad = self.probs.clone()
        grad[torch.arange(self.batch_size), self.y_true] -= 1
        grad /= self.batch_size
        return grad

```

Листинг 5 - Реализация класса контейнера модели.

```

class Model:
    def __init__(self, layers):
        self.layers = layers

```

```

def forward(self, x):
    for layer in self.layers:
        x = layer.forward(x)
    return x

def backward(self, grad):
    for layer in reversed(self.layers):
        grad = layer.backward(grad)
    return grad

def parameters(self):
    params = []
    for layer in self.layers:
        if hasattr(layer, "W"):
            params.append((layer.W, layer.dW))
            params.append((layer.b, layer.db))
    return params

def predict(self, x):
    with torch.no_grad():
        logits = self.forward(x)
        probs = torch.softmax(logits, dim=1)
        return torch.argmax(probs, dim=1)

```

Листинг 6 - Реализация класса оптимизатора Adam.

```

class Adam:
    def __init__(self, params, lr=0.001, beta1=0.9, beta2=0.999,
eps=1e-8):
        self.params = params
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
        self.eps = eps

        self.t = 0
        self.m = []
        self.v = []

        for param, _ in self.params:
            self.m.append(torch.zeros_like(param))
            self.v.append(torch.zeros_like(param))

    def step(self):
        self.t += 1

```

```

        for i, (param, grad) in enumerate(self.params):
            self.m[i] = self.beta1 * self.m[i] + (1 - self.beta1) * grad
            self.v[i] = self.beta2 * self.v[i] + (1 - self.beta2) *
(grad ** 2)

            m_hat = self.m[i] / (1 - self.beta1 ** self.t)
            v_hat = self.v[i] / (1 - self.beta2 ** self.t)

            param -= self.lr * m_hat / (torch.sqrt(v_hat) + self.eps)

    def zero_grad(self):
        for _, grad in self.params:
            grad.zero_()

```

Листинг 7 - Реализация цикла обучения моделей.

```

import torch

def get_batches(X, y, batch_size, shuffle=True):
    n = X.shape[0]
    indices = torch.arange(n)

    if shuffle:
        indices = indices[torch.randperm(n)]

    for start in range(0, n, batch_size):
        end = start + batch_size
        batch_idx = indices[start:end]
        yield X[batch_idx], y[batch_idx]

def accuracy(logits, y_true):
    preds = torch.argmax(logits, dim=1)
    return (preds == y_true).float().mean().item()

def train(model, criterion, optimizer, X_train, y_train, X_test, y_test,
epochs=10, batch_size=64):
    optimizer.params = model.parameters()
    optimizer.step()
    train_losses = []
    train_accs = []
    test_accs = []

```

```

for epoch in range(epochs):
    epoch_loss = 0.0
    epoch_acc = 0.0
    batches = 0

    # train
    for x_batch, y_batch in get_batches(X_train, y_train,
batch_size):
        logits = model.forward(x_batch)
        loss = criterion.forward(logits, y_batch)

        grad = criterion.backward()
        model.backward(grad)

        optimizer.params = model.parameters()
        optimizer.step()
        optimizer.zero_grad()

        epoch_loss += loss
        epoch_acc += accuracy(logits, y_batch)
        batches += 1

    train_losses.append(epoch_loss / batches)
    train_accs.append(epoch_acc / batches)

    # test
    with torch.no_grad():
        logits_test = model.forward(X_test)
        test_acc = accuracy(logits_test, y_test)
        test_accs.append(test_acc)

    print(
        f"Epoch {epoch+1}/{epochs}: | "
        f"Loss: {train_losses[-1]:.4f} | "
        f"Train acc: {train_accs[-1]:.4f} | "
        f"Test acc: {test_acc:.4f}"
    )

return train_losses, train_accs, test_accs

```


Эксперименты

Для экспериментов были выбраны 4 архитектуры моделей. Первая модель - базовая нейронная сеть с двумя полносвязными линейными слоями и функцией активации ReLU. Вторая - совпадает по архитектуре с первой, но имеет больше нейронов. В третьей добавляется еще один скрытый слой. Четвёртая совпадает по архитектуре с третьей, но так же имеет больше нейронов.

Далее на листингах 8 - 11 представлены результаты обучения моделей, а на рисунках 1-3 графики изменения метрик моделей в течение обучения.

Листинг 8 - Результат обучения первой модели.

Треним первую модель

Epoch 1/10:	Loss: 0.3863	Train acc: 0.8945	Test acc: 0.9404
Epoch 2/10:	Loss: 0.1722	Train acc: 0.9502	Test acc: 0.9598
Epoch 3/10:	Loss: 0.1214	Train acc: 0.9648	Test acc: 0.9669
Epoch 4/10:	Loss: 0.0916	Train acc: 0.9733	Test acc: 0.9702
Epoch 5/10:	Loss: 0.0723	Train acc: 0.9786	Test acc: 0.9726
Epoch 6/10:	Loss: 0.0585	Train acc: 0.9826	Test acc: 0.9739
Epoch 7/10:	Loss: 0.0475	Train acc: 0.9860	Test acc: 0.9757
Epoch 8/10:	Loss: 0.0393	Train acc: 0.9881	Test acc: 0.9757
Epoch 9/10:	Loss: 0.0325	Train acc: 0.9905	Test acc: 0.9770
Epoch 10/10:	Loss: 0.0270	Train acc: 0.9923	Test acc: 0.9766

Листинг 9 - Результат обучения второй модели.

Треним вторую модель

Epoch 1/10:	Loss: 0.3293	Train acc: 0.9107	Test acc: 0.9502
Epoch 2/10:	Loss: 0.1335	Train acc: 0.9615	Test acc: 0.9678
Epoch 3/10:	Loss: 0.0875	Train acc: 0.9737	Test acc: 0.9710
Epoch 4/10:	Loss: 0.0651	Train acc: 0.9802	Test acc: 0.9780
Epoch 5/10:	Loss: 0.0494	Train acc: 0.9852	Test acc: 0.9775
Epoch 6/10:	Loss: 0.0381	Train acc: 0.9884	Test acc: 0.9786
Epoch 7/10:	Loss: 0.0293	Train acc: 0.9918	Test acc: 0.9795
Epoch 8/10:	Loss: 0.0232	Train acc: 0.9932	Test acc: 0.9763
Epoch 9/10:	Loss: 0.0180	Train acc: 0.9949	Test acc: 0.9807
Epoch 10/10:	Loss: 0.0145	Train acc: 0.9961	Test acc: 0.9778

Листинг 10 - Результат обучения третьей модели.

Треним третью модель

Epoch 1/10:	Loss: 0.4851	Train acc: 0.8553	Test acc: 0.9249
Epoch 2/10:	Loss: 0.1942	Train acc: 0.9430	Test acc: 0.9553
Epoch 3/10:	Loss: 0.1279	Train acc: 0.9625	Test acc: 0.9631
Epoch 4/10:	Loss: 0.0957	Train acc: 0.9706	Test acc: 0.9674
Epoch 5/10:	Loss: 0.0755	Train acc: 0.9768	Test acc: 0.9724
Epoch 6/10:	Loss: 0.0603	Train acc: 0.9809	Test acc: 0.9723
Epoch 7/10:	Loss: 0.0499	Train acc: 0.9839	Test acc: 0.9702
Epoch 8/10:	Loss: 0.0408	Train acc: 0.9872	Test acc: 0.9748
Epoch 9/10:	Loss: 0.0333	Train acc: 0.9892	Test acc: 0.9709
Epoch 10/10:	Loss: 0.0264	Train acc: 0.9910	Test acc: 0.9709

Листинг 11 - результат работы четвертой модели.

Треним четвертую модель

Epoch 1/10:	Loss: 0.4236	Train acc: 0.8727	Test acc: 0.9390
Epoch 2/10:	Loss: 0.1616	Train acc: 0.9526	Test acc: 0.9609
Epoch 3/10:	Loss: 0.1004	Train acc: 0.9703	Test acc: 0.9699
Epoch 4/10:	Loss: 0.0721	Train acc: 0.9774	Test acc: 0.9691
Epoch 5/10:	Loss: 0.0535	Train acc: 0.9833	Test acc: 0.9756
Epoch 6/10:	Loss: 0.0422	Train acc: 0.9871	Test acc: 0.9797
Epoch 7/10:	Loss: 0.0311	Train acc: 0.9901	Test acc: 0.9785
Epoch 8/10:	Loss: 0.0278	Train acc: 0.9908	Test acc: 0.9783
Epoch 9/10:	Loss: 0.0207	Train acc: 0.9934	Test acc: 0.9821
Epoch 10/10:	Loss: 0.0174	Train acc: 0.9944	Test acc: 0.9795

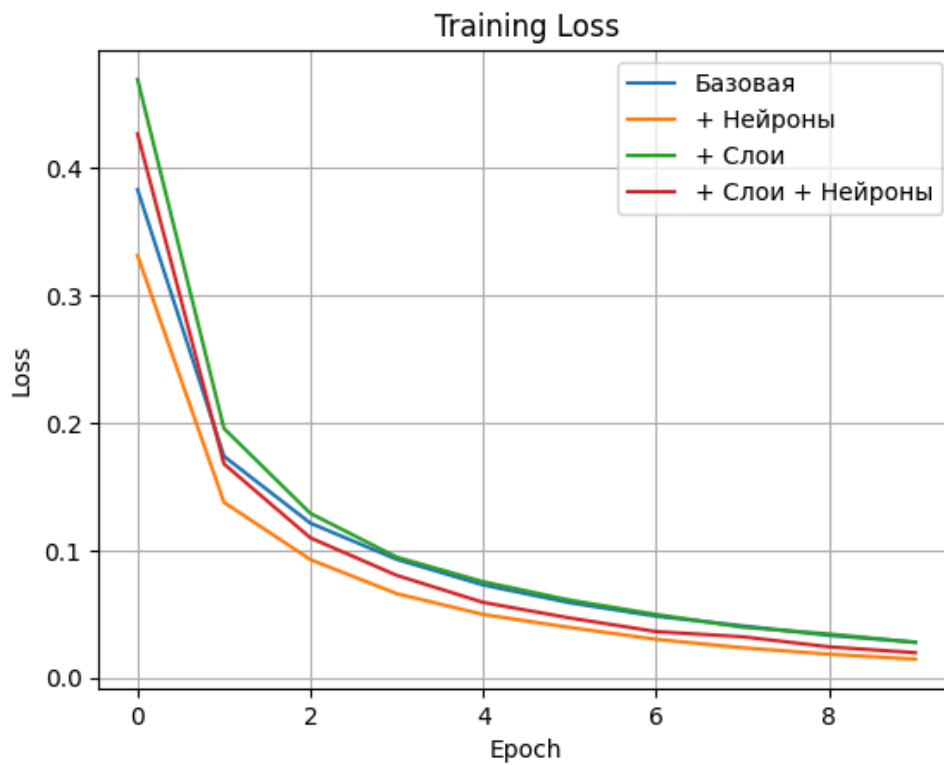


Рисунок 1 - График изменения Loss моделей в процессе обучения.

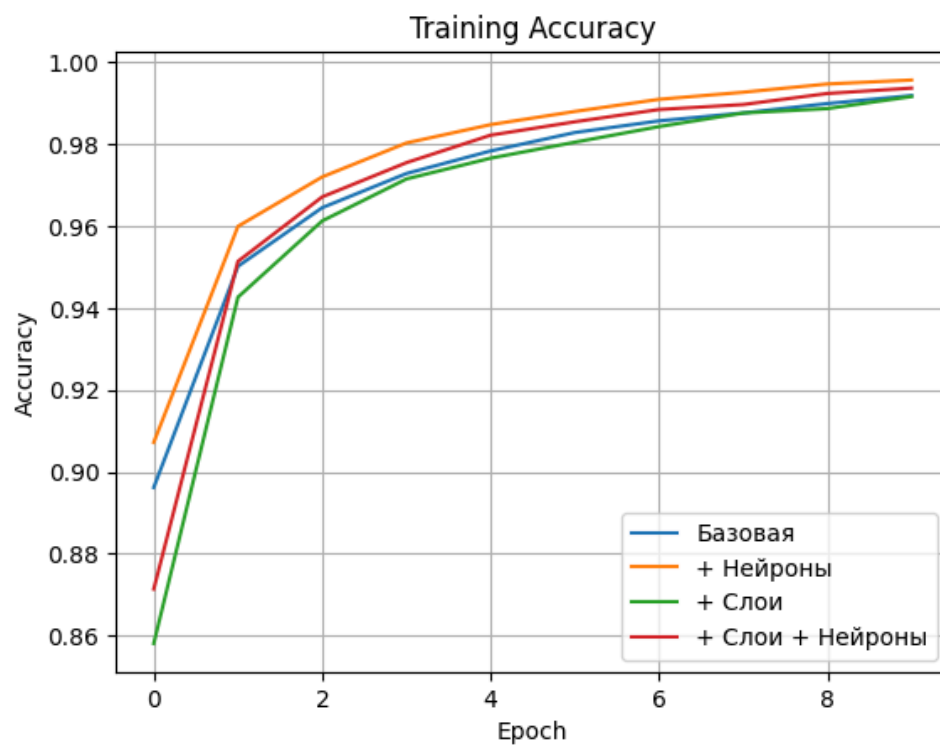


Рисунок 2 - График изменения Accuracy моделей в процессе обучения.

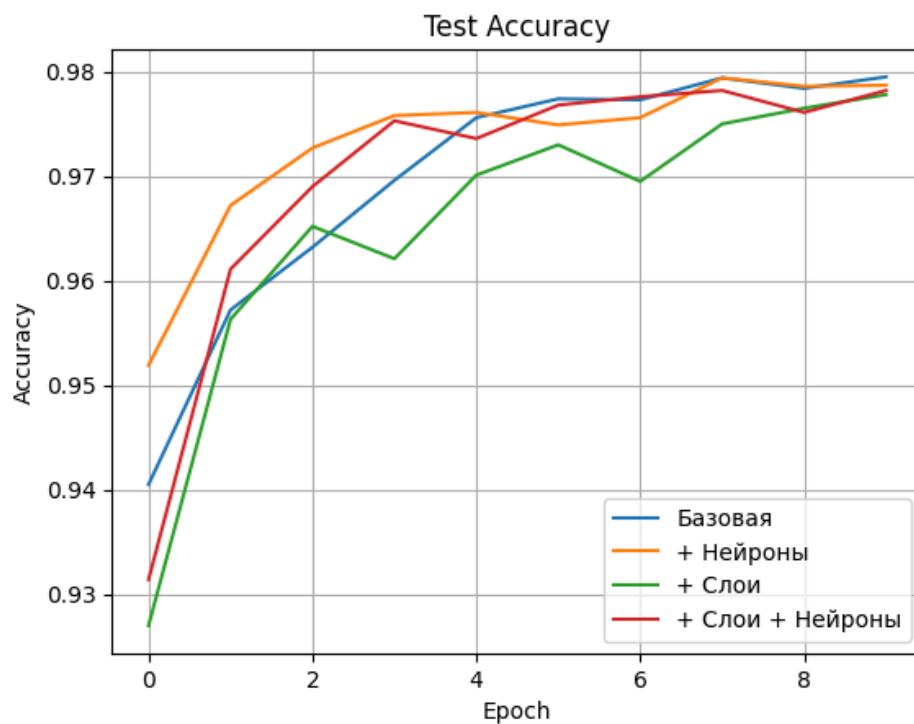


Рисунок 3 - График изменения Ассигасу моделей на тестовой выборке.

Наилучшие результаты по итогам обучения показали вторая (“+ Нейроны”) и четвертая (“+ Слои + Нейроны”) модели.

Далее на рисунках 4-13 представлены графики ROC- кривых для каждого класса.

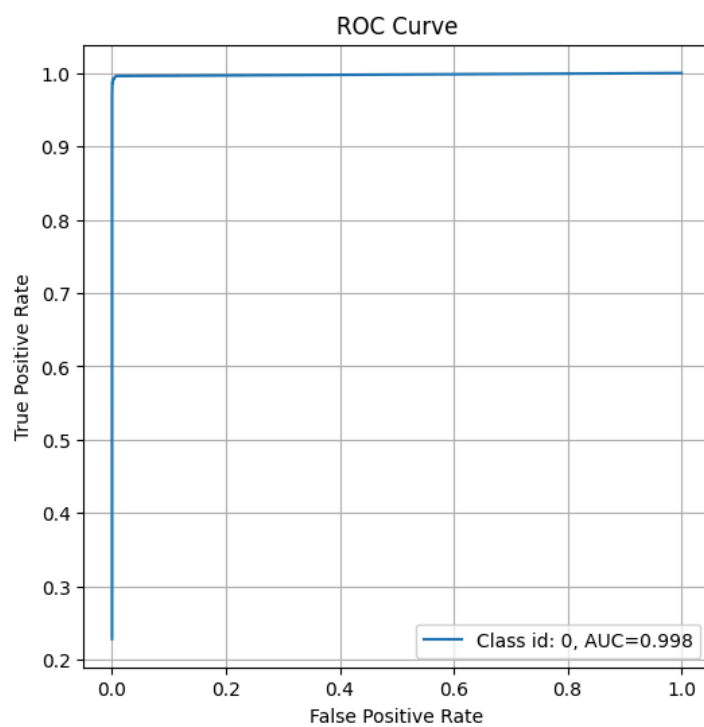


Рисунок 4 - ROC-кривая для класса “0”.

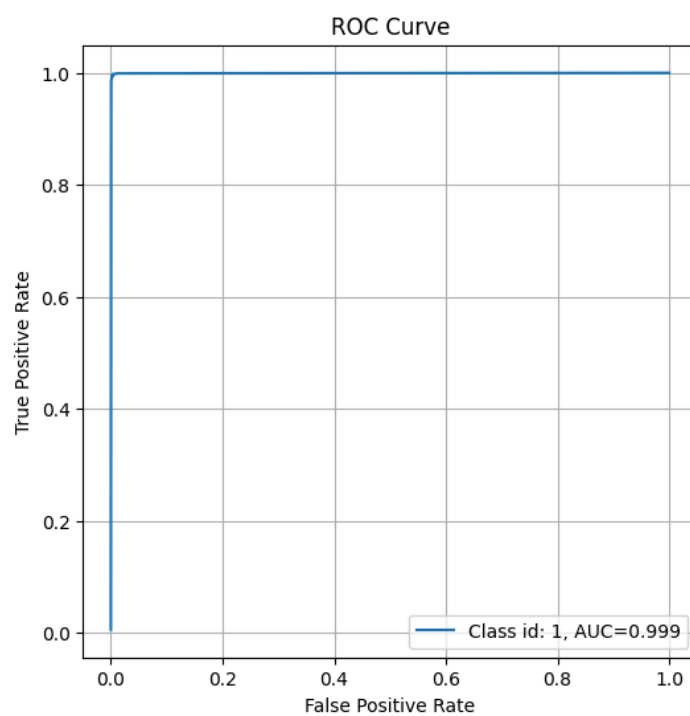


Рисунок 5 - ROC-кривая для класса “1”.

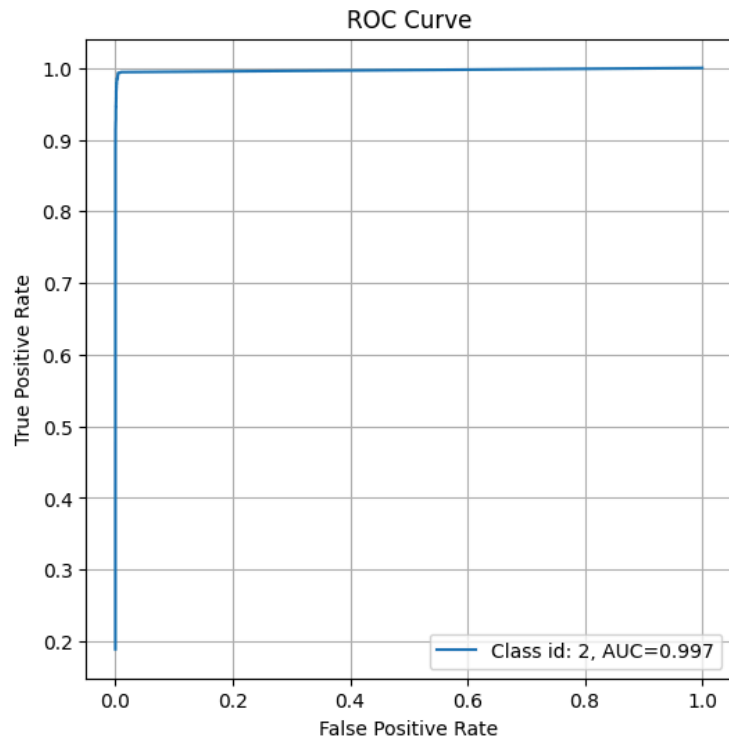


Рисунок 6 - ROC-кривая для класса “2”.

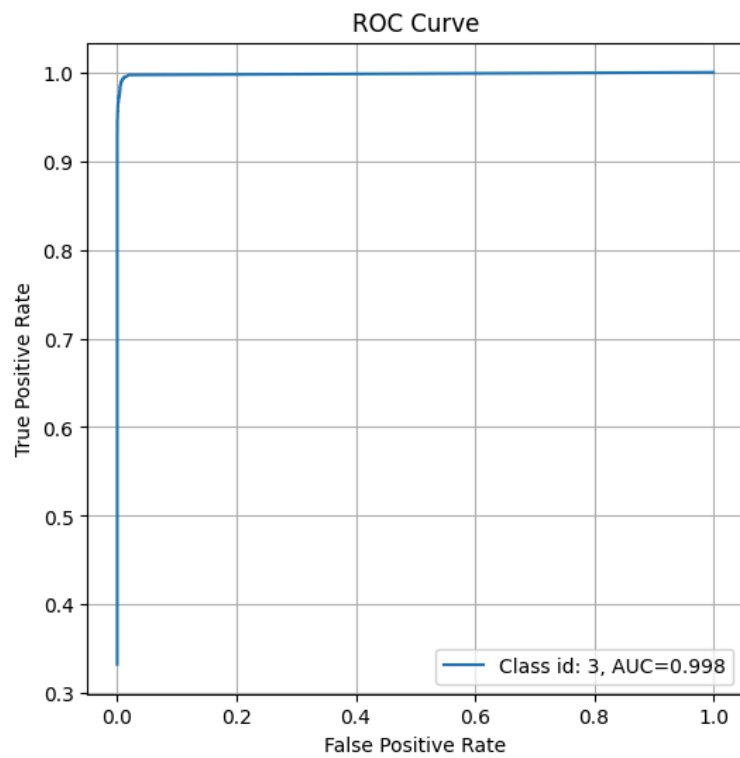


Рисунок 7 - ROC-кривая для класса “3”.

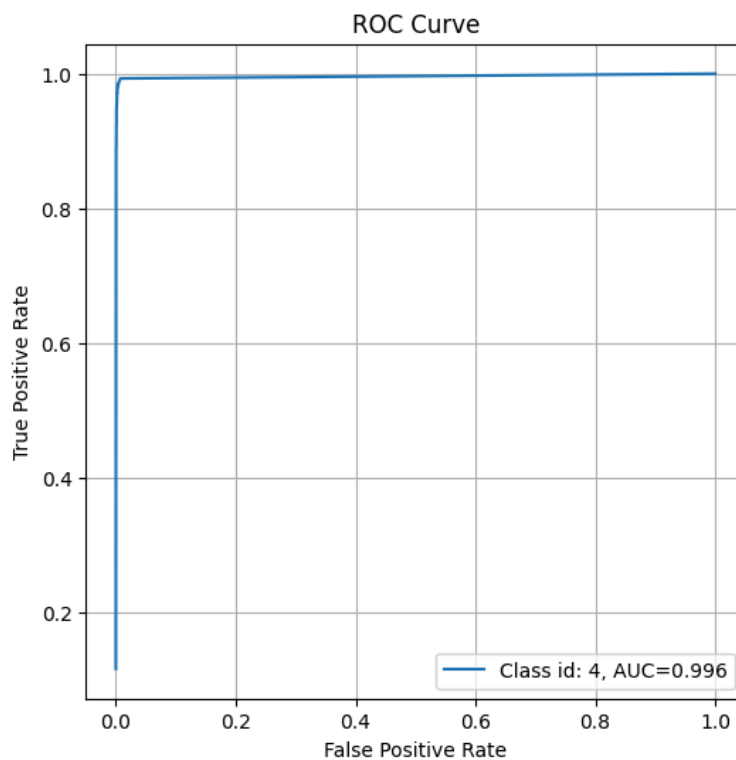


Рисунок 8 - ROC-кривая для класса “4”.

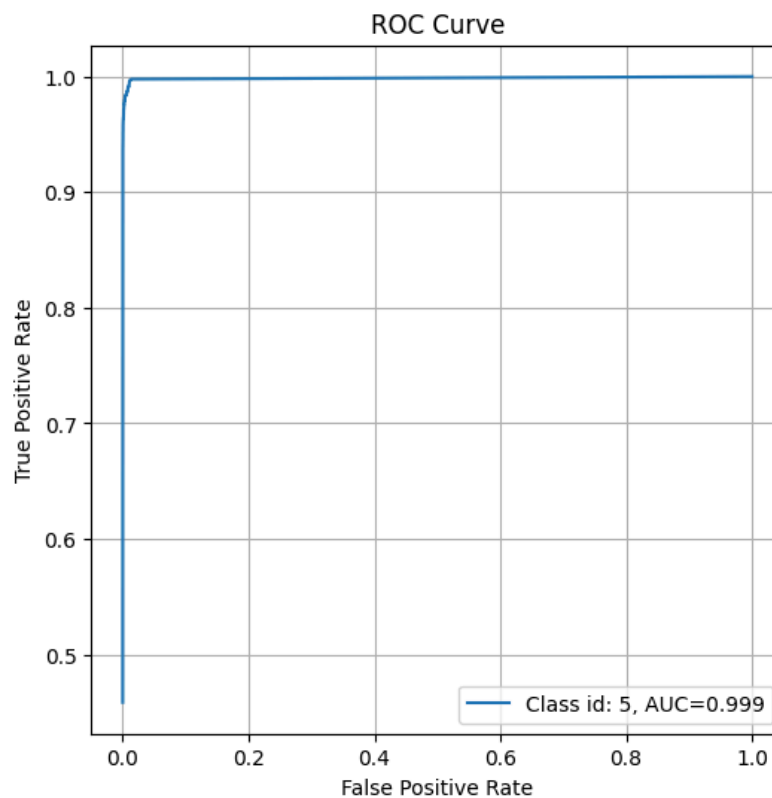


Рисунок 9 - ROC-кривая для класса “5”.

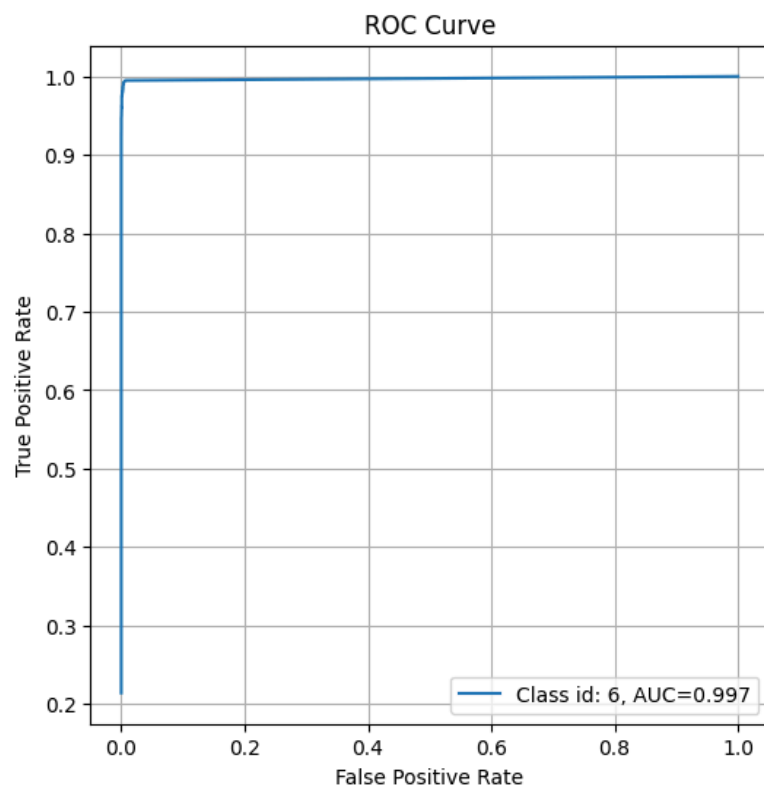


Рисунок 10 - ROC-кривая для класса “6”.

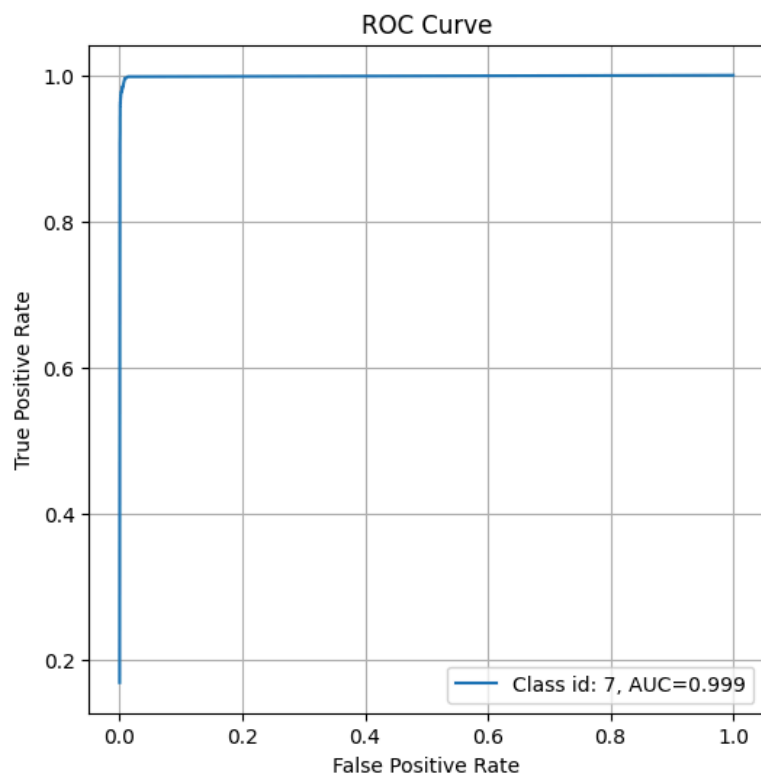


Рисунок 11 - ROC-кривая для класса “7”.

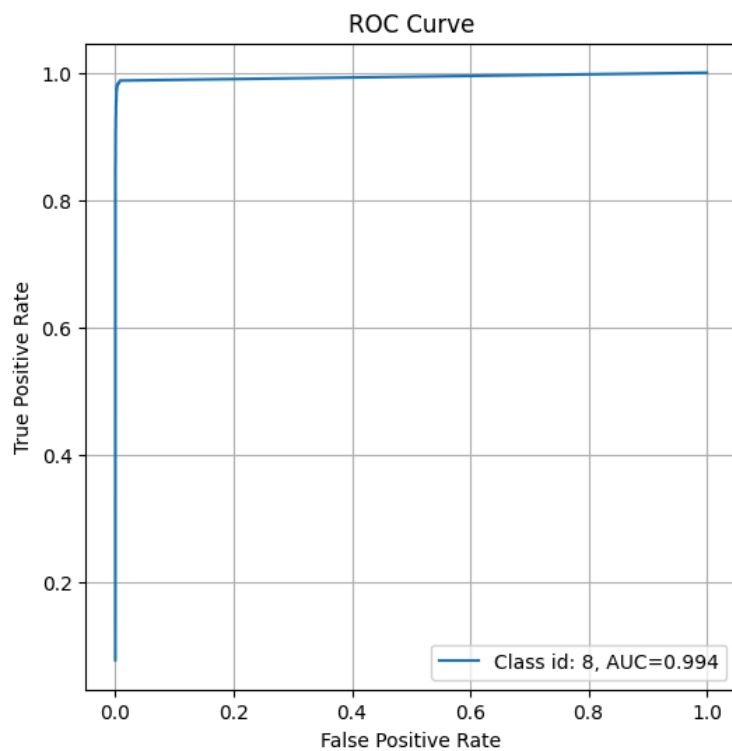


Рисунок 12 - ROC-кривая для класса “8”.

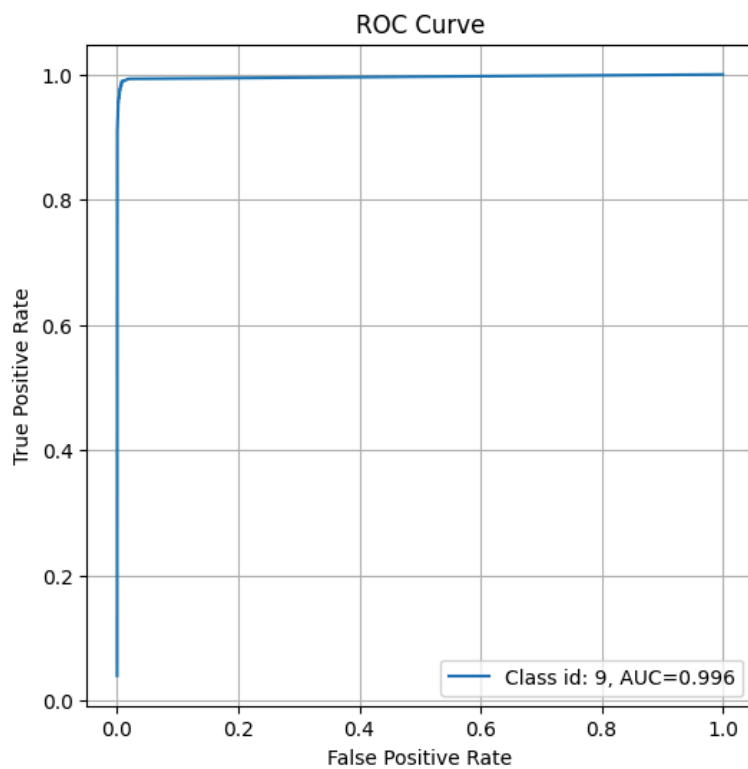


Рисунок 13 - ROC-кривая для класса “9”.

Вывод

В ходе выполнения лабораторной работы были написаны несколько классов слоёв для нейронных сетей, обучены несколько моделей с различными архитектурами на реальных данных. Были проведены эксперименты над моделями и они были протестированы на своих образцах.

Приложение

Исходный код лабораторной работы представлен по ссылке:
<https://colab.research.google.com/drive/1pxIJDJq5c5jiqkMLIQXK5v6sKRVaauIf?usp=sharing>